Hierarchical Abstract Tree for Cross-Document Retrieval-Augmented Generation

Ziwen Zhao [1] **Menglin Yang** [1]

Abstract

Retrieval-augmented generation (RAG) enhances large language models with external knowledge, and tree-based RAG organizes documents into hierarchical indexes to support queries at multiple granularities. However, existing Tree-RAG methods designed for single-document retrieval face critical challenges in scaling to cross-document multi-hop questions: *(1) poor distribution adaptability*, where k-means clustering introduces noise due to rigid distribution assumptions; *(2) structural isolation*, as tree indexes lack explicit cross-document connections; and *(3) coarse abstraction*, which obscures fine-grained details. To address these limitations, we propose **Ψ-RAG**, a tree-RAG framework with two key components. *First*, a hierarchical abstract tree index built through an iterative "merging and collapse" process that adapts to data distributions without a priori assumption. *Second*, a multi-granular retrieval agent that intelligently interacts with the knowledge base with reorganized queries and an agent-powered hybrid retriever. Ψ-RAG supports diverse tasks from token-level question answering to document-level summarization. On cross-document multi-hop QA benchmarks, it outperforms RAPTOR by 25.9% and HippoRAG 2 by 7.4% in average F1 score[1].

1. Introduction

Retrieval-augmented generation (RAG) (Lewis et al., 2020) enhances large language models (LLMs) with reliable references from external knowledge bases. To effectively address multi-granular user queries from token-level factual question answering (QA) to document-level summarization, RAG frameworks must leverage the semantic hierarchy within corpora (Sarthi et al., 2024; Chen et al., 2024). Traditional retrieval systems (Robertson et al., 2009; Karpukhin et al., 2020) slice documents into short, independent chunks; this fixed retrieval granularity leads to incomplete and inaccurate responses for higher-level questions (Nair et al., 2023; Liu et al., 2021; Sarthi et al., 2024; Wang et al., 2025).

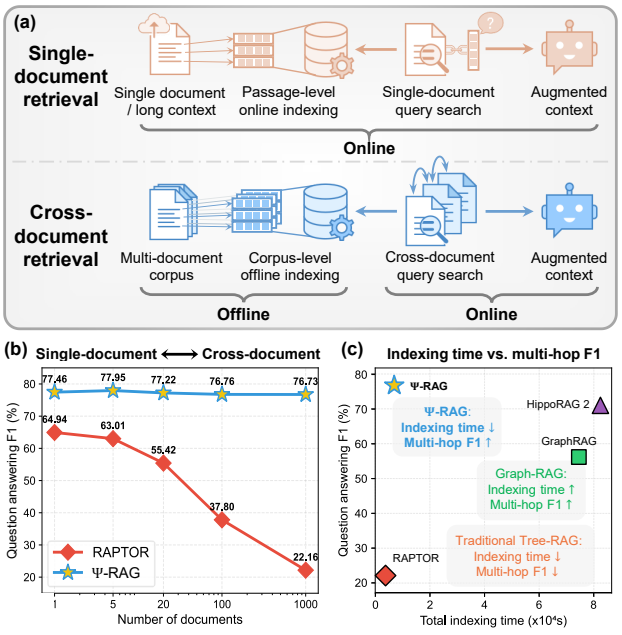

Figure 1. (a) Two RAG application scenarios: single-document and cross-document retrieval. (b) Multi-hop QA performance *w.r.t.* the number of documents in the tree index. (c) Ψ-RAG has both efficient indexing and accurate multi-hop retrieval compared to structured RAG baselines, using 2Wiki as an example.

Structured RAG achieves multi-granularity by organizing documents relationally with knowledge graph indexes (Edge et al., 2024; Jiménez Gutiérrez et al., 2024; 2025) or hierarchically with tree indexes (Liu et al., 2021; Jin et al., 2025b; Sarthi et al., 2024). Its utility is demonstrated in two key scenarios, as in Figure 1(a): (1) *single-document retrieval*, to process user requests in diverse granularity within a single user-uploaded document (Jin et al., 2025a; Li et al., 2025b; Zhang et al., 2025a); (2) *cross-document retrieval*, to process user requests by building a large index from an entire

[1]The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China. Correspondence to: Menglin Yang <menglinyang@hkust-gz.edu.cn>.

Proceedings of the 43^{rd} International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026.

[1]Code is available at https://github.com/Newiz430/Psi-RAG.

Table 1. Comparison of RAG frameworks. "?" indicates limited support by specific methods.

Task type	Single-document	Cross-document	Token-level	Passage-level	Document-level
Traditional RAG	✓	×	✓	?	×
Graph-RAG	✓	✓	✓	✓	?
Tree-RAG	✓	×	?	✓	✓
Ψ-RAG (Ours)	✓	✓	✓	✓	✓

(domain-specific) corpus and retrieving across multiple documents, challenging the multi-hop retrieval and reasoning capabilities of RAG frameworks (Trivedi et al., 2023; Tang & Yang, 2024).

Our work focuses on *Tree-RAG* (Liu et al., 2021; Jin et al., 2025b; Sarthi et al., 2024; Tao et al., 2025; Rezazadeh et al., 2025), which organizes documents into a tree-structured index. It explicitly defines different information granularities, enabling superior retrieval performance, especially for summative tasks (Sarthi et al., 2024; Xiao et al., 2025). See Appendix F for more related works. However, existing Tree-RAG methods mainly target single-hop retrieval on a single long document, while cross-document and multi-hop retrieval remain underexplored. For example, representative k-means-type cluster trees like RAPTOR (Sarthi et al., 2024) achieve high retrieval performance only with small-scale, passage-level indexes. As shown in Figure 1(b), **RAPTOR's retrieval accuracy drops significantly when the search space expands to corpus-level with millions of tokens**. We identify three key limitations: (1) *Poor distribution adaptability*: k-means-type clustering implicitly relies on spherical data distributions (Sarthi et al., 2024), which introduces noisy documents to the retriever for corpora with a skewed distribution. (2) *Structural isolation*: unlike Graph-RAG which dynamically hops between documents based on pairwise relationships, leaf nodes in a tree index lack explicit connections. This prevents the retriever from capturing implicit causal dependencies in multi-hop questions. (3) *Coarse abstraction*: the coarse-grained abstracts act like a mosaic, obscuring token-level details at the very beginning of retrieval, as dense vector matching struggles to precisely associate a specific entity in the user query with abstract concepts on top of the tree.

This work aims to address the limitations of Tree-RAG on cross-document and multi-hop retrieval. We present Ψ-RAG, a versatile and effective Tree-RAG framework, comprising a *hierarchical abstract tree index* and a *multi-granular agentic retriever*. (1) To address poor distribution adaptability, we build an abstract tree index via a hierarchical clustering-inspired "merging and collapse" process, which iteratively links similar chunks to existing or newly generated upper-level nodes in the tree. (2) To tackle structural isolation, a retrieval and answering agent reasons through the user query, during which it calls for additional document retrieval with a reorganized query when needed. This empowers the tree retriever with better causal understanding of user query. (3) To resolve the conflict between coarse abstraction and detailed factual retrieval, a multi-granular hybrid retrieval framework is employed, where an agent-powered sparse retriever supplements the hierarchical tree retrieval with fine-grained information, yielding significant performance gains. Our proposed framework possesses numerous advantages:

- **Ψ-RAG generalizes Tree-RAG from passage-level to corpus-level indexing.** As shown in Figure 1(c), Ψ-RAG builds a corpus-level tree 10× faster than OpenIE-based Graph-RAG. Under this setting, Ψ-RAG achieves average gains of 23.7% in retrieval and 25.9% in generation over RAPTOR on token-level QA.
- **Ψ-RAG extends Tree-RAG to complex multi-hop scenarios.** To our knowledge, Ψ-RAG is the first Tree-RAG for cross-document multi-hop scenarios comparable to Graph-RAG, exhibiting great application potential. As in Figure 1(c), Ψ-RAG is both efficient and powerful, outperforming cutting-edge Graph-RAG frameworks like HippoRAG 2 (Jiménez Gutiérrez et al., 2025).
- As in Table 1, **Ψ-RAG is an all-in-one Tree-RAG framework supporting tasks across various granularities**: token-level factual QA, passage-level causal reasoning, and document-level summarization.
- **Ψ-RAG is built entirely with open-source LLMs**. Its components are flexibly replaceable. Ψ-RAG can be applied to custom corpora without any training or fine-tuning, demonstrating strong generalizability.

2. Preliminary

Retrieval-Augmented Generation. Given a user query q, an LLM $f(q; I)$ generates a corresponding response a, where I is the system instruction for a specific task. For a reasoning model, $a = R \cup y$, where R is the Chain-of-Thought (Wei et al., 2022) reasoning text and y is the final answer. An LLM equipped with RAG can be defined as $f \cdot r$, where the retriever $r(q; \mathcal{I})$ maps q to top-k relevant document chunks $\mathcal{D}^* = \{u_i\}_{i=1}^k \subset \mathcal{D}$ using a scoring function $s(q, u)$. The external corpus $\mathcal{D}$ contains $n = |\mathcal{D}|$ chunks in total. The index $\mathcal{I} : \mathcal{D} \to \mathcal{T}$ maps each chunk in $\mathcal{D}$ to a discrete index space $\mathcal{T}$ for better retrieval.

Tree Index. We overload the symbol $\mathcal{T}$ to denote a tree. Formally, a tree $\mathcal{T} = (\mathcal{V}, \mathcal{E})$ is a connected acyclic graph with a finite node set $\mathcal{V} = \{v_i\}_{i=1}^{|\mathcal{V}|}$ and an edge set $\mathcal{E} \subset \mathcal{V} \times \mathcal{V}$. The root node for a node u is denoted as $root(u)$. The set of children of u is $c(u) = \{v | u \to v \in \mathcal{E}\}$. Let $\ell(u)$ be the set of all leaf nodes reachable from u. The depth of a node $\delta(\cdot)$ is defined as the length of the path to the root,